

Aula 2 - Compose e Volumes

Docupedia Export

Author:Cardoso Cristian (CtP/ETS)

Date:27-May-2026 13:24

Table of Contents

1 Docker compose e volumes	3
1.1 Volume	3
1.2 Compose	3
1.3 Prática	3
1.3.1 Dockerfile	3
1.3.2 DockerCompose	4
1.3.3 Services	7
2 Comandos Docker Compose	11
2.1 Up	11
2.2 PS (Process Status)	11
2.3 Down	11
2.4 Stop	11
2.5 Start	12
2.6 Logs	12
2.7 EXEC	12
3 Atividades	13
3.1 API Panetone	13
3.1.1 POST /Task	13
3.1.2 GET /Task	13
3.2 Atividade 1	14
3.3 Atividade 2	14
3.4 Atividade 3	14
3.5 Atividade 4	14

1 Docker compose e volumes

1.1 Volume

É um diretório dentro do nosso linux server detinado ao armazenamento de arquivos, ele não utiliza imagem nem container, pelo contrário, containers podem usar de um volume para guardar seus arquivos, se não o fizerem, tudo que aconteceu desde quando o container foi criado baseado na imagem até então vai ser descartado, usamos **volumes** para não perder dados de uma instância de container para outra.

1.2 Compose

Assim como o **Dockerfile** criaremos um arquivo com nome específico, "**docker-compose.yml**". Docker compose é um arquivo para configuração de diferentes containers mas que possuem relação entre si, como por exemplo definir que nosso container principal vai utilizar de um volume.

1.3 Prática

Vamos subir uma aplicação .NET para um container agora:

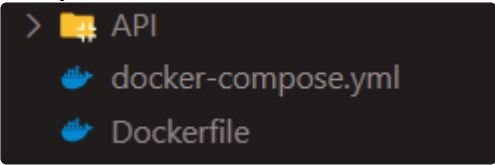
- Iniciar a aplicação

```
dotnet new webapi -n <NOME>
```

Agora criaremos nossos arquivos para o Docker:

- Dockerfile
- docker-compose.yml

Ou seja:



```
> API
  docker-compose.yml
  Dockerfile
```

1.3.1 Dockerfile

Definiremos nosso Dockerfile, para uma aplicação .NET9 vamos utilizar a seguinte imagem:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0
```

Montamos nossa área de trabalho:

```
WORKDIR /app
```

Podemos pensar no nosso container como um máquina nova e zerada dentro da rede Bosch, portanto é importante definir também as variáveis de ambiente. Como utilizamos o **RB Local Proxy Manager** a variável de ambiente deve ser o equivalente a "<http://localhost:3128>", entretanto se tentarmos utilizar "<http://localhost>" dentro do container ele vai achar que é sobre o kernel linux, quando na realidade queremos nos referir a máquina principal (host), então precisamos substituir "<http://localhost>" por: "host.docker.internal"

```
ENV HTTP_PROXY="host.docker.internal:3128"  
ENV HTTPS_PROXY="host.docker.internal:3128"
```

Copiar os arquivos do projeto (com boas práticas):

```
COPY ./API/*.csproj ./  
RUN dotnet restore  
COPY ./API ./
```

Comando para quando o container iniciar:

```
CMD ["dotnet", "run", "--urls=http://0.0.0.0:4040"]
```

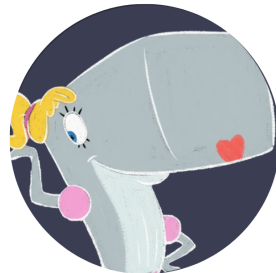
1.3.2 DockerCompose

Agora vamos entender como funciona o *docker-compose.yml*.

Para iniciar vamos esclarecer o que é um Docker Compose, ele é basicamente o maestro da orquestra, quem coordena e faz com que diferentes instrumentos (containers) trabalhem juntos a fim de gerar uma música final (aplicação completa)



Uma breve explicação:
Docker explicado por garotas!



Vamos entender como declarar um **docker-compose.yml**, ele utiliza **YAML**, uma linguagem baseada em indentação para definir hierarquias. Diferente do JSON, não usa chaves `{ }` nem vírgulas, a estrutura é definida apenas pelo alinhamento.

A lógica funciona assim:

```
bloco1:
  bloco11:
    atributo11: "valor"
  bloco12:
    bloco121:
  block13:
    bloco131:
      bloco1311:
    atributo12: "valor"

bloco2:
  atributo21: "valor"
  bloco21:
    bloco211:
      atributo2111: "valor"
      atributo2111: valor"
  atributo22: "valor"
```

Portanto é possível perceber que existe uma hierarquia nas definições, aplicando para a sintaxe correta do Docker Compose, podemos utilizar de 6 blocos para o **root-level**.

- **name** - Define o nome do projeto
- **services** - Define os containers da aplicação
- **volumes** - Define volumes utilizados para persistência de dados
- **networks** - Define redes personalizadas para comunicação entre containers
- **configs** - Permite injetar arquivos de configuração nos containers
- **secrets** - Gerenciamento seguro de dados sensíveis (senhas, tokens)

1.3.3 Services

Dentro do services podemos criar um bloco com nome qualquer, esse nome será o nome do serviço criado, dentro do bloco do nosso serviço podemos iniciar ele baseado em uma imagem ou então em um Dockerfile.

Baseado em Dockerfile, utilizamos **"build"**, e passamos como valor o local que o Dockerfile que será usado está:

```
services:
  api:                # -> <NOME_DO_SERVICO>
    build: .          # -> <CAMINHO_DO_DOCKERFILE>
```

Ou de forma expandida, pois por padrão o docker-compose.yml procurará por um arquivo chamado "Dockerfile", mas como voce pode alterar o nome, voce deve especificar aqui:

```
services:
  <NOME_DO_SERVICO>:  # -> <NOME_DO_SERVICO>
    build:
      context: .       # -> <CAMINHO_DO_DOCKERFILE>
      dockerfile: elifrekcoD  # -> <NOME_DO_DOCKERFILE>
```

Baseado em uma imagem utilizamos **"image"**, e passamos o link da imagem desejada

```
services:
  api:                # -> <NOME_DO_SERVICO>
    image: mcr.microsoft.com/dotnet/sdk:9.0
```

Com **"environment"** podemos declarar variáveis de ambiente do sistema do container, criar ou definir

```
services:
  api:
    environment:
      APP_ENV: Production
```

Utilizando **"ports"** podemos declarar em que porta do host vamos expor qual porta do container, por exemplo, se sua aplicação está configurada para rodar na porta 5000, ela vai rodar na 5000 do container, então devemos dizer que a porta 5000 exposta do container vai rodar em alguma porta do host, podendo ser a 5000 também ou qualquer outra disponível, no caso abaixo exemplificado o container roda na 5000 mas expõe através da 8080 do host, ou seja, sua aplicação local precisará ser acessada através de **http://localhost:8080**:

```
services:
```

```
api:
  ports:
    - 8080:5000
    # -> HOST:CONTAINER
```

"**volumes**", usado para definir basicamente que, quando o container for usar tal pasta, na realidade vai mandar para o volume definido. Entretanto, como demonstrado no exemplo abaixo, o nome do volume só funcionará se o volume for definido, para criar um volume precisamos voltar para **root keys**, dentro da root key **volumes**, declaramos o nosso volume:

```
services:
  api:
    volumes:
      - logs:app/logs
      # -> <NOME_DO_VOLUME>:<CAMINHO_LINKADO>

volumes:
  logs
```

Outra maneira importante de sobre o uso da key **volumes** dentro de um serviço é o link direto entre arquivos, ou seja, posso dizer que quando o container tentar acessar arquivo X na realidade ele vai usar o arquivo Y, para isso definimos da seguinte forma:

```
services:
  api:
    volumes:
      - ./arquivo.txt:/etc/arquivos/arquivo.txt
      # -> <CAMINHO DO ARQUIVO>:<CAMINHO_DO_ARQUIVO_LINKADO>
```

existem algumas opções para o controle de acesso aos arquivos, chamadas de **flags de montagem**, por padrão quando não é colocado nada o docker entende que é "**rw**" (Read-Write), mas temos outras opções:

- "**ro**" - Read-Only, ou seja, o container não vai conseguir editar o arquivo
- "**z**" - Indica que esse volume é usado por vários containers
- "**Z**" - Indica que esse volume é exclusivo desse container

Uso:


```

services:
  api:
    volumes:
      - ./arquivo.txt:/etc/arquivos/arquivo.txt:ro      # -> <CAMINHO DO ARQUIVO>:<CAMINHO_DO_ARQUIVO_LINKADO>
      - logs:app/logs:z                                  # -> <NOME_DO_VOLUME>:<CAMINHO_LINKADO>
volumes:
  logs

```

Sobre "**networks**", para iniciarmos vamos entender conceitualmente o básico sobre networks, criamos uma "rede", para definirmos as conexões internas dos nossos containers, por exemplo, com minha API e meu banco de dados na mesma rede posso dizer que a API irá se conectar ao banco e expor a porta apenas da API. Se não for declarado "ports", não é possível acessarmos a partir do host, portanto podemos declarar as portas expostas apenas da API, deste modo nós acessamos a API mas não conseguimos acessar o banco, entretanto a API por estar na mesma rede do banco pode acessá-lo. Para podermos utilizar dessas networks precisamos defini-las na **root key** "networks".

Para a API acessar o banco pode ser feito da seguinte forma:

<NOME_DO_SERVICO_DO_BANCO>:<PORTA_QUE_O_BANCO_ESTA_RODANDO_INTERNAMENTE>, essa informação deve ser passada através da connection string, exemplo para uma string de conexão mssql:

Server=<NOME_DO_SERVICO_DO_BANCO>,<PORTA_QUE_O_BANCO_ESTA_RODANDO_INTERNAMENTE>;Database=<DATABASE>;User Id=<USER>;Password=<PASSWORD>;TrustServerCertificate=True

Caso não seja declarado uma network para o serviço por padrão o compose insere todos os serviços sem network em uma mesma rede default

```

services:
  api:
    networks:
      - backend
    ports:
      - 8080:8080
    environment:
      DB_HOST: db      # -> <NOME_DO_SERVIÇO_DO_BANCO>
      DB_PORT: 6767    # -> <PORTA_INTERNA_NO_CONTAINER_DO_BANCO>
  db:
    networks:
      - backend

networks:
  backend:

```

Com isso dentro do nosso container da API podemos declarar a string de conexão se baseando nas variáveis de ambiente definidas acima.

"**depends_on**", é usado para definir a ordem de inicialização, por exemplo, nossa API não deve rodar antes do nosso banco, se não quando tentarmos nos conectar vai falhar, portanto a API depende do banco de dados, e isso pode ser declarado da seguinte forma:

```
services:
  api:
    depends_on:
      - db
  db:
```

"**env_file**", pode ser usado para definir variáveis de ambiente baseadas em um arquivo .env

```
services:
  api:
    env_file:
      - .env
```

Hora do pensamento 🤔

Como faremos para o front-end acessar o nosso back-end, considerando que queremos evitar que ele acesse o banco, portanto o front deve estar em uma rede separada, como faremos a conexão correta?

```
=====
||                                     ||
||  COLOCAR A RESPOSTA AQUI DEPOIS  ||
||                                     ||
=====
```

2 Comandos Docker Compose

2.1 Up

Para subir o compose, e caso nunca tenha sido buildado builda pela primeira vez utilizamos o seguinte comando:

```
docker compose up
```

Caso queira subir o compose e ja iniciar todos os containers é necessário passar a flag "-d",

```
docker compose up
```

E se quiser rebuildar, caso tenha tido alterações no docker-compose.yml é necessário utilizar "--build", isso irá forçar o compose a rodar novamente, caso contrário vai continuar utilizando a configuração anterior

```
docker compose up --build
```

É possível utilizar ambos simultaneamente

```
docker compose up -d --build
```

2.2 PS (Process Status)

Para visualizar os containers do nosso compose, se estão rodando, em que porta estão expostos, quando subiram, etc

```
docker compose ps
```

2.3 Down

derruba permanentemente todo o compose, encerrando todos os containers, e se passado a flag "-v" até os volumes podem ser encerrados definitivamente.

```
docker compose down
```

2.4 Stop

Parar compose sem remover

```
docker compose stop
```

2.5 Start

Iniciar o que foi parado

```
docker compose start
```

2.6 Logs

Visualizar os logs de atividade do compose

```
docker compose logs
```

Podendo ser especificado a qual serviço gostaria de ver os logs, entra em modo de monitoramento desse container:

```
docker compose logs -f <NOME_DO_SERVICO>
```

2.7 EXEC

Execução dentro de um container, executar comandos via terminal bash (lembrando que é um ambiente linux)

```
docker compose exec <NOME_DO_SERVICO> <COMANDO>
```

Exemplo que irá mostrar todas as variáveis de ambiente do sistema:

```
docker compose exec api env
```

3 Atividades

Utilize a API fornecida para a realização das tarefas

3.1 API Panetone

3.1.1 POST /Task

Cadastra uma nova tarefa.

Requisição:

```
{
  "Name" : "Name",
  "Description" : "Description",
  "Responsible" : "Responsible",
  "Status" : "Status"
}
```

Resposta:

```
panetone
```

3.1.2 GET /Task

Retorna todas as tarefas cadastradas

Requisição: Sem body

Resposta:

```
{
  "data": {
    "tasks": []
  },
  "isSuccess": true,
  "reason": ""
}
```

3.2 Atividade 1

Monte o Dockerfile e o DockerCompose apenas para a API, e observe o que irá ocorrer

3.3 Atividade 2

Adicione o banco de dados SQL Server aos serviços

- Suba seu compose
- Cadastre três novas tarefas
- Encerre seu compose
- Suba-o novamente
- Busque por todas as tarefas

3.4 Atividade 3

Utilize um .env para definir as variáveis de ambiente da API

3.5 Atividade 4

Adicione um **volume** ao seu banco de dados e então:

- Suba seu compose
- Cadastre três novas tarefas
- Encerre seu compose
- Suba-o novamente
- Busque por todas as tarefas